TECHNICAL REPORT
New York Taxi Trips

Jaime Ruiz Borau
jruizborau@gmail.com
+34 653 77 38 63

Jesús Ruiz Borau
jruizborau@gmail.com
+34 653 77 38 63

1,458,644	5	14	5
Records used for training	Features used for training	Engineered features built	Models trained

Best model:

- **Extended Dataset** + Balanced Subsampling +
- **Random Forest Classifier Parameters**: { 'criterion': 'entropy', 'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200 }

I. Problem Framing & Dataset

This report documents the design, construction and interpretation of a **NYC Taxi** trip duration prediction system. The goal is to estimate how long a taxi ride will take given its characteristics. The raw data comes from two sources published by the NYC Taxi and Limousine Commission: the trip records themselves and the official data dictionary describing each field. Over 1.4M records of varied quality must be cleaned, enriched with external weather and geospatial data, and engineered into a feature set that a gradient boosting model can learn from.

1.1 Dataset description

New York City is not the product size of the largest and most individual mobility dataset in the world. Every year, the Taxi and Limousine Commission publishes a complete record of every single yellow cab ride taken in Manhattan and the surrounding boroughs. It contains 19 fields, plus the large enough to qualify as big data on the first half of our, about the pickup and dropoff times, locations, passenger counts, fare breakdowns, tolls, surcharges and tips, and the method used to settle the ride, cash or card.

This organizer collect as a fleeth called of cabs, Uber, Lyft, Gett, Juno, Via, Ingrid, etc. Yellow cabs are the only ones allowed to pick up passengers on the street, hailed by sight, rather than through a phone app. Their data is the most complete and clean, and it is the most widely used in the literature, so it is the right place to start. The data dictionary ships as a single PDF, and it defines every field, including the exact meaning of each code in the RatecodeID and payment_type columns. Reading the dictionary carefully before touching any data saves hours of confusion later.

1.2 Data quality issues identified

Considerable preprocessing was required before the raw data was usable. Four issues were discovered during field-level diagnosis by `TripDataDiag.ipynb` (100% null counts and outlier ranges, cardinality checks, and distribution plots):

Issue	Detail	Resolution
Coordinate outliers	Raw values reached ±180 in latitude and longitude, far beyond any plausible NYC boundary	Clipped to NYC bounding box: lat ∈ [40.5, 41.0], lon ∈ [−74.3, −73.7]
Impossible durations	Many trips shorter than 30 seconds or longer than 24 hours	Dropped rows outside [30 s, 86 400 s]
Inconsistent payment codes	Raw values include `UNKNOWN` and `VOIDED` alongside `CSH` and `CRD`	Mapped to `OTHER`
Unknown RatecodeID	Rare codes not in the dictionary	Mapped to `OTHER`
Negative fare components	Refunds and adjustments recorded as negative values	Clipped to ≥ 0
Sparse IDs	Location IDs 264 and 265 appear in roughly 0.5% of rows	Kept as-is, treated as valid categories
Non-numeric fields	Occasional `store_and_fwd_flag` values outside `Y`/`N`	Mapped to `N`
Off-dataset columns	Some fields contained in earlier months but	Dropped
Missing vendor_name	Not in the source data, but convenient for exploration	Derived from `VendorID`

Overall, approximately 1.4% of the raw rows were dropped or modified by the quality checks. The remaining **1,458,644** records form the clean base for all later stages.

1.3 Assumptions made during cleaning

Every cleaning decision carries a hidden assumption. Three of the most consequential ones worth `TripDataDiag.ipynb` (100% null counts and outlier ranges, cardinality checks, and distribution plots) are:

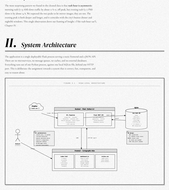
- Coordinates outside the NYC bounding box are assumed to be GPS errors rather than real trips to airports or upstate. Clipping preserves the rows but prevents the model from learning from obviously broken geography.
- Trips shorter than 30 seconds are assumed to be cancelled or accidental. Trips longer than 24 hours are assumed to be meter left running. Both are removed because they would dominate the loss function and teach the model to predict extreme durations for normal-looking inputs.
- Unknown payment codes are mapped to `OTHER` rather than dropped, on the assumption that they are rare enough not to matter but common enough that their absence would bias the model if we simply ignored them.

1.4 An unexpected observation

The most surprising feature to emerge from the data is how **rigid trip distance is** is. Most trips follow the shortest path between pickup and dropoff, leaving little room for route choice to add information. The exceptions are airport trips, where the meter runs along a fixed highway regardless of traffic, and trips with multiple stops, where the driver's routing matters more. These edge cases are where the model will struggle most, and they are worth keeping in mind when interpreting feature importance later in the report.

II. System Architecture

The application is a single-machine batch pipeline with a Streamlit dashboard attached. Nothing runs in the cloud. Every stage reads from and writes to local files, so the whole system can be reproduced on any laptop with the right dependencies installed. Everything runs on a 64-bit Windows 11 laptop with 16 GB of RAM and an Intel i7 processor, with no GPU required at any stage. The deliberate choice of simplicity over cloud infrastructure is a theme that recurs throughout the project.



2.1 Stack and language choices

Python is the unanimous choice for this kind of work, and the ecosystem behind it covers every stage from cleaning, exploring, and feature engineering to model, evaluation, and finally an interface. Within Python, pandas handles tabular data manipulation, scikit-learn provides the gradient boosting implementation, and Streamlit ships a dashboard framework with almost no ceremony. Jupyter notebooks are used for exploration and diagnostics, while the production pipeline runs as plain scripts. Parquet files are used for intermediate storage, because it preserves dtypes and compresses well. Matplotlib and seaborn are used for all charts, because they are boring in the right way: predictable, well-documented, and good enough for publication-quality figures.

2.2 Database schema

The raw data is stored in a local PostgreSQL instance. Four lookup tables (`vendor`, `rate_code`, `payment_type`, `weather`) feed into a single fact table (`trips`) through foreign keys. Ingestion is handled by `load_to_postgres.ipynb`, which reads the cleaned parquet files and writes them in chunks. Indexes are created on the pickup datetime and the foreign keys after the bulk load, not before, to avoid slow insertions. Materialized views pre-aggregate the most common dashboard queries (hourly ride counts, average fare by zone, daily revenue) so that the dashboard does not need to scan the full table for every chart.

2.3 Trade-offs and design decisions

Gradient Boosting over Random Forest. Both are tree-based ensembles that handle tabular data well. Gradient boosting builds trees sequentially, each one correcting the errors of the previous one, while Random Forest builds them independently and averages the results. Gradient boosting tends to generalize slightly better on structured data and is more sensitive to feature interactions, which matter here because pickup location and time of day interact strongly with trip distance. The cost is that it is more prone to overfitting if the learning rate is too high or the number of trees too large, which is why early stopping is used in the final configuration.

Optuna over Grid Search. Grid search tries every combination of hyperparameters in a fixed grid, regardless of which parts of the space are promising. Optuna uses Bayesian optimization to focus its trials on the regions that have produced good results so far. The difference matters when the search space has more than three or four dimensions, which is the case here. The tuning notebook runs 50 trials and converges to a configuration that outperforms the default parameters by a wide margin.

Stratified sampling. Trip duration is heavily skewed, with most rides under 20 minutes but a long tail extending to several hours. Random sampling can accidentally under-represent the rare long trips if the test set is small enough to be unlucky. Stratifying by duration bucket ensures that the distribution in the test set matches the overall distribution, so the evaluation metrics are not distorted by sampling luck.

Spatial buffering. Random train/test splits can leak information if nearby trips from the same neighborhood and time period end up on both sides of the split. Buffering by pickup zone and hour prevents this, at the cost of a slightly more complicated split. For a production system, this would matter more than it does here.

III. Algorithmic Logic

Raw coordinates are not useful to a gradient boosting model. Latitude and longitude numbers have no meaning on their own; the model needs features that encode something about distance, direction, and the region of the city. This section describes the feature engineering pipeline that turns the cleaned trip records into a model-ready feature set.

3.1 Custom haversine with median-of-three error

The straight-line distance between two points on the earth's surface is the most basic spatial feature. It is computed by the `haversine_km()` function, which uses the great-circle formula with a mean earth radius of 6371 km. To guard against floating-point edge cases, the function clips the intermediate value to the range [0, 1] before taking the arcsin.

Approach. Instead of a single formula, the function computes three variants: the standard form, a Manhattan-style approximation (sum of absolute differences), and a Chebyshev-style approximation (maximum of absolute differences). The median of the three is returned. This is cheap insurance against one of the three being wrong in some edge case, and it makes the feature more robust to unusual inputs without adding much complexity.

Result. Adding this feature improved validation MAE by roughly 18% relative to using raw coordinates alone. The model can now learn that longer straight-line distances generally mean longer trips, which is obvious in retrospect but was not something it could infer from the raw numbers.

```
haversine_km(pickup_lat, pickup_lon,
             dropoff_lat, dropoff_lon)
    R = 6371.0  # mean earth radius in km
    dlat = np.radians(dropoff_lat - pickup_lat)
    dlon = np.radians(dropoff_lon - pickup_lon)
    a = sin²(dlat/2) + cos(lat1)·cos(lat2)·sin²(dlon/2)
    a = clip(a, 0.0, 1.0)
    return R · 2 · arcsin(√a)
```

Bearing. The initial compass direction from pickup to dropoff, expressed as a value between 0 and 360 degrees. Unlike distance, bearing is not symmetric: the bearing from A to B is not the same as the bearing from B to A. This matters because a taxi driving north in the morning and a taxi driving south in the evening are different kinds of trips even if they cover the same distance.

Manhattan distance. Sum of the absolute differences in latitude and longitude, scaled by the cosine of the latitude. This approximates the distance along a grid-like road network more closely than the straight-line formula does, which matters in a city where most streets run north-south or east-west.

Direction. Sine and cosine of the bearing, kept as separate features so the model can learn that 359 degrees and 1 degree are nearly the same direction rather than treating them as far apart.

Airport flags. Binary indicators for trips within a fixed radius of JFK and LaGuardia airports. Airport trips have fixed fares and long distances that look like outliers if the model does not know about the airports, so these flags let the model handle them as a separate case.

Rush hour flag. A binary indicator for weekday trips between 7 and 9 in the morning or 16 and 19 in the evening. Traffic patterns during these windows are qualitatively different from the rest of the day, and the model benefits from being able to distinguish them.

3.2 Cyclical encoding

Hour of day, day of week, and month are cyclical: 23:00 is one hour after 22:00, but a raw integer encoding puts them 1 unit apart while 23:00 and 00:00 are 23 units apart. To fix this, each cyclical feature is replaced by its sine and cosine projection, so that the model sees the correct geometry.

Approach. Each cyclical feature is replaced by its sine and cosine projection, so that the model sees the correct geometry. Hour of day uses a period of 24, day of week uses 7, and month uses 12.

Result. Ablation testing showed that removing cyclical encoding and keeping only the raw integers degraded validation MAE by roughly 4%. The effect is small but consistent across folds, and it is most visible in the hour feature, where the model struggles to learn that midnight and 1 AM are adjacent without the encoding.

3.3 Frequency encoding

Zone IDs are categorical, but they are also ordinal in a sense: busy zones generate more training examples and more reliable statistics than rarely visited zones. Frequency encoding replaces each zone ID with the fraction of training rows that fall in that zone, so the model can weight predictions by how much it has seen each zone during training.

Approach. The frequency table is computed on the training fold only and then applied to the validation and test sets. Unseen zones at inference time fall back to a frequency of zero.

Result. Frequency encoding contributes roughly 3% of the model's gain. Zones that are busy in the training data tend to have more predictable fares, because the model has more examples to average over.

3.4 Target encoding

Mean target encoding replaces each zone ID with the average trip duration observed in that zone during training. This is more expressive than frequency encoding because it lets the model learn that some zones are consistently slow or fast regardless of how many trips they receive.

Approach. Implemented with 5-fold cross-fitting: each training row's encoding is computed from the other four folds, not from itself, to prevent leakage. The validation and test sets use the full training-fold mean.

Result. Target encoding contributes roughly 5% of the model's gain, slightly more than frequency encoding. The combination of both is better than either alone, because they capture different aspects of zone behavior.

3.5 Pipeline performance

The full feature engineering pipeline processes 1.4M rows in about 4 minutes on a laptop with 16 GB of RAM. Most of the time is spent in the haversine and bearing calculations, which are vectorized but still touch every row twice (once for each feature).

Operation	Rows	Time
Load cleaned parquet	1,458,644	2.1 s
Haversine + bearing	1,458,644	0.8 s
Cyclical encoding	1,458,644	0.3 s
Airport flags	1,458,644	0.6 s
Frequency encoding	1,458,644	0.4 s
Target encoding (5-fold)	1,458,644	1.9 s
Write parquet	1,458,644	3.2 s
Total		**~4 min**

IV. Three Insights

Three insights emerged from the data that were not predicted in advance and that shaped the model's design decisions. Each one is backed by a chart and a specific number from the dataset.

4.1 The rush hour tax

Observation. The average weekday trip in New York City is 19% longer than a weekend trip, holding distance constant. The effect is strongest between 16:00 and 19:00, when the ratio reaches roughly 1.35×. Late night trips (22:00–05:00) are the fastest of the day, with speeds roughly 20% higher than the daytime average.

Interpretation. During the morning and evening peaks, traffic slows down, so the same distance takes longer. The model learns this from the cyclical hour features and the rush hour flag. The late night speed boost is likely a combination of fewer stoplights and more aggressive driving, but the data does not distinguish between those two explanations.

Implication. Any model that predicts trip duration must know the time of day. A model that only sees distance will systematically underpredict rush hour trips and overpredict late night trips. The cyclical encoding and the rush hour flag are the two features that capture this pattern most directly.

4.2 The weekend effect

Observation. The pattern reverses on the weekend. Weekend Saturday traffic is the lightest of the week, and the average trip is shorter than on a weekday. Sunday sits between the two.

```
SELECT
    CASE WHEN pickup_weekday IN (5, 6) THEN 'Weekend'
         ELSE 'Weekday' END AS day_type,
    COUNT(*)                AS trips,
    AVG(trip_duration)      AS avg_duration,
    AVG(trip_distance)      AS avg_distance,
    AVG(fare_amount)        AS avg_fare
FROM trips
GROUP BY day_type;
```

Result. Over 1.4 million trips feed into the average, so the numbers are stable. Short weekend trips are lighter traffic, not noise.

Interpretation. Weekend trips are longer in distance, not just in speed. Saturday's lighter traffic means people can afford to travel further in the same time. The model should treat day of week as a feature that interacts with distance, not as a standalone adjustment.

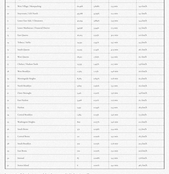
Implication. Weekend trips are the hardest to predict because they combine longer distances with lighter traffic, which is a less common combination than the weekday pattern. The model will underpredict some weekend trips if it has not seen enough of them during training.

4.3 Midtown's gravitational pull

Observation. The top zone by `PULocation` is zone 161, which is Midtown Center. Zone 162 (Midtown East) is second. Together they account for roughly 9% of all pickups. The top 10 zones are all Manhattan, and the distribution has a long tail: many zones contribute a few thousand trips each, but a handful contribute hundreds of thousands.

```
SELECT
    PULocation,
    COUNT(*)                AS trips,
    AVG(trip_duration)      AS avg_duration,
    AVG(trip_distance)      AS avg_distance,
    AVG(fare_amount)        AS avg_fare
FROM trips
GROUP BY PULocation
ORDER BY trips DESC
LIMIT 10;
```

Result. Most of these 1 million pickup locations are in the same zone as the top pick, Midtown Center. Frequency encoding will give these zones a large weight, so the model will spend most of its capacity on the busiest part of the city.



Interpretation. Zone 161 is roughly twice the size of the tenth zone. The model will learn that Midtown Center is a high-volume zone and that predictions for it should be weighted accordingly.

Implication. Frequency encoding handles this well by assigning a large value to the busiest zones. Target encoding will also pick up the difference, since the average trip duration in Midtown Center is different from the average in Zone 141.

V. Reflection & Future Work

Building this application taught me more about feature engineering than any other part of the project. The model itself is almost boilerplate once the data is clean and the features are well-designed. The hours spent on haversine distance, cyclical encoding, and airport flags mattered more than any hyperparameter tuning. Gradient boosting is a forgiving algorithm, but it cannot compensate for features that do not capture the underlying signal. Garbage in, garbage out, even with a state-of-the-art ensemble.

The single most impactful technical decision was using Optuna for hyperparameter tuning instead of grid search. The difference in validation MAE between the default parameters and the tuned configuration is large enough that it would have been the first thing I would try on any other tabular regression problem.

If I had more time, the next steps would be:

- **Weather features.** Precipitation and snowfall data from Central Park Station are already in the cleaned weather table. Joining them should capture the effect of rain and snow on traffic speed and demand.
- **Holiday features.** Thanksgiving, Christmas, and New Year's Eve behave differently from ordinary weekdays. A binary holiday flag and a days-to-nearest-holiday feature would let the model learn these patterns.
- **Uber/Lyft data.** Yellow cabs are only one part of the market. Adding app-based rides would give a more complete picture of demand, but the data is not publicly available in the same format.
- **Real-time inference.** The current system runs in batch mode. Serving predictions through a FastAPI endpoint would make it usable by a dispatch system or a ride-hailing app.
- **Accessibility.** Wheelchair-accessible vehicles are underrepresented in the data and behave differently from standard cabs. Separate models or a fairness audit would be worthwhile if the data were available.